

Module 1 – Foundation

1. What is a HTTP?

HTTP (Hyper Text Transfer Protocol) is a communication protocol used to transfer data between a web browser and a web server. It helps load web pages on the internet.

2. What is a Browsers? How they works?

A **browser** (like Google Chrome or Mozilla Firefox) is software used to access websites.

How it works:

1. You enter a URL
2. Browser sends a request to the server
3. Server sends back data (HTML, CSS, JS)
4. Browser displays the webpage

3. What is Domain Name?

A **domain name** is the address of a website (e.g., google.com). It helps users easily access websites instead of using IP addresses.

4. What is hosting?

Hosting is a service that stores your website files on a server and makes them accessible on the internet 24/7.

Fundamentals of World Wide Web

1. Difference between Web Designer and Web Developer .

Web Designer:

A web designer focuses on the **visual appearance and user experience (UI/UX)** of a website. Their job is to make the website attractive and easy to use. They decide layout, colors, fonts, images, and overall design. Tools like Figma, Photoshop, and basic HTML/CSS are often used.

Web Developer:

A web developer focuses on the **functionality and working** of the website. They write code to make the website interactive and dynamic. Developers use technologies like HTML, CSS, JavaScript, and backend languages (PHP, Node.js, etc.).

In short:

Designer = looks of website

Developer = working of website

2. What is a W3C?

(W3C) is an international organization that develops **standards and guidelines for the web**.

Its main goal is to ensure that all websites work properly across different browsers and devices. It defines standards for technologies like HTML, CSS, and XML.

For example, when developers follow W3C standards, websites work correctly on browsers like Google Chrome and Mozilla Firefox.

3. What is Domain?

A **domain** (or domain name) is the **unique address of a website** on the internet, such as google.com or facebook.com.

Instead of remembering complex IP addresses (like 192.168.1.1), users can simply type a domain name. The domain name system (DNS) converts the domain into an IP address so the browser can find the website.

4. What SEO?

SEO (Search Engine Optimization) is the process of improving a website so that it appears higher in search engine results like Google.

SEO includes:

- Using proper keywords
- Improving website speed
- Writing quality content
- Making the site mobile-friendly
- Getting backlinks

The main goal of SEO is to **increase website traffic and visibility**.

5. What is SDLC life cycle?

SDLC (Software Development Life Cycle) is a structured process used to develop high-quality software step by step.

Main phases of SDLC:

1. **Planning:** Identify project goals and requirements
2. **Analysis:** Study user needs and system requirements
3. **Design:** Create system architecture and UI design
4. **Development:** Write and build the code
5. **Testing:** Check for bugs and errors
6. **Deployment:** Launch the software for users
7. **Maintenance:** Fix issues and update the system

Purpose:

SDLC helps in developing software in a **systematic, efficient, and error-free way**.

Fundamentals of IT

1. What is a Program?

- **Explain in your own words what a program is and how it functions.**

A **program** is a set of instructions written in a programming language that tells a computer **what to do and how to do it**.

How it functions:

1. The programmer writes the program using a language (like C, Java, or JavaScript).
2. The code is then **compiled or interpreted** into machine language that the computer understands.
3. When the program runs, the computer follows the instructions step by step.
4. It takes input (if needed), processes it, and produces output.

2. What is Programming?

- **What are the key steps involved in the programming process?**

The **programming process** involves a series of steps to create a working and efficient program:

1. Problem Definition :

Understand the problem clearly—what needs to be solved and what the expected output is.

2. Planning (Algorithm & Flowchart) :

Create a step-by-step solution (algorithm) and sometimes a flowchart to visualize the logic before coding.

3. Coding :

Write the program using a programming language like C, Java, or JavaScript based on the planned logic.

4. Compilation/Execution :

Convert the code into machine-readable form and run it to check how it works.

5. Testing and Debugging :

Test the program with different inputs and fix errors (bugs) to ensure correct output.

6. Documentation :

Write comments and explanations so others (and you later) can understand the code easily.

7. Maintenance :

Update and improve the program after deployment (fix bugs, add features, improve performance).

3. Types of Programming Languages?

There are two types of programming languages : **1. High-level** and **2. Low-level**.

- **What are the main differences between high-level and low-level programming languages?**

1. Low-Level Languages

- Very close to machine code
- Difficult for humans to understand

- Faster execution speed
- Direct interaction with hardware
- Examples: Machine Language, Assembly Language

2. High-Level Languages

- Easy to read and write (English-like syntax)
- Requires compiler or interpreter
- Slower than low-level languages
- Portable across different systems
- Examples: C, Java, Python, JavaScript

4. World Wide Web & How Internet Works?

The **World Wide Web (WWW)** is a collection of interconnected web pages accessed through the internet.

Working of Internet:

1. User enters a website URL
2. DNS converts domain name into IP address
3. Browser sends request using HTTP
4. Server receives and processes request
5. Server sends response (HTML, CSS, JS)
6. Browser renders and displays the webpage

- **Describe the roles of the client and server in web communication.**

Client:

- A device or software (like a web browser)
- Sends requests to the server
- Displays the received data

Server:

- A computer/system that stores data
- Processes client requests
- Sends responses back to client

5. Network Layers on Client and Server

1. Application Layer
2. Transport Layer
3. Internet Layer
4. Network Access Layer

- **Explain the function of the TCP/IP model and its layers.**

The **TCP/IP model** is a set of rules that governs how data is transmitted over the internet.

Functions:

- Breaks data into packets
- Sends packets over network
- Ensures delivery to correct destination
- Reassembles data at receiver side

Layers of TCP/IP Model:

1. **Application Layer**
 - User interaction
 - Examples: HTTP, FTP
2. **Transport Layer**
 - End-to-end communication
 - TCP (reliable), UDP (faster)
3. **Internet Layer**
 - Logical addressing (IP)
 - Routing of packets
4. **Network Access Layer**
 - Physical data transmission
 - Includes hardware and network media

6. Client and Servers

Client

A **client** is a device or software that **requests services or data** from a server.
Examples: Web browsers like

Server

A **server** is a computer or system that **provides data or services** to clients. It stores websites, files, or databases and responds to client requests.

- **Explain Client Server Communication**

Client-server communication is the process in which a client sends a request and the server sends back a response.

Steps of Communication

1. **User Action**
The user enters a URL in the browser
2. **Request Sent**
The client sends an HTTP request to the server
3. **Server Processing**
The server receives the request and processes it
4. **Response Sent**
The server sends back data (HTML, CSS, JS)
5. **Display Output**
The client (browser) displays the webpage

7. Types of Internet Connections

1. **Dial-up** – Very slow, uses telephone line
2. **Broadband** – High-speed (DSL, Cable, Fiber)
3. **Wireless** – No cables (Wi-Fi, Mobile Data like 4G/5G)
4. **Satellite** – Used in remote areas
5. **Leased Line** – Fast and dedicated (used by businesses)

- **How does broadband differ from fiber-optic internet ?**

Broadband vs Fiber-Optic Internet

- **Broadband**
 - General term for high-speed internet
 - Includes DSL, cable, and fiber
 - Uses copper wires or coaxial cables
 - Speed varies (moderate to high)
- **Fiber-Optic Internet**
 - A type of broadband

- Uses fiber-optic cables (light signals)
- Very high speed and low latency
- More reliable and faster than other broadband types

8. Protocols – Difference between HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is used to transfer data between a browser and a server, but it is not secure because the data is sent in plain text. This means attackers can read or modify the data.

HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP. It uses encryption (SSL/TLS) to protect the data during transmission. This ensures privacy, data integrity, and authentication.

In simple terms, HTTP is not secure, while HTTPS is secure and recommended for all websites.

9. Application Security – Role of Encryption

Encryption plays a major role in securing applications by converting readable data into an unreadable format. Only authorized users with the correct key can decrypt and read the data.

It protects sensitive information such as passwords, personal data, and financial details from hackers. Encryption ensures confidentiality, data integrity, and secure communication between users and systems.

Software Applications and Its Types

Software applications are programs designed to perform specific tasks for users.

Types of software applications include:

- General-purpose software like word processors and web browsers
- Specialized software like accounting or design tools
- Web applications that run in browsers
- Mobile applications designed for smartphones

Difference between System Software and Application Software

- System software manages and controls the computer hardware and provides a platform for other software. Examples include operating systems like Windows.
- Application software is designed to perform specific user tasks such as writing documents, browsing the internet, or editing images.
- In simple words, system software runs the computer, while application software helps users perform tasks.

10. Software Architecture – Significance of Modularity

- Modularity means dividing a software system into smaller, independent modules.
- It is important because it makes the system easier to develop, test, and maintain. Each module can be updated or fixed without affecting the entire system. It also improves code reusability and teamwork, as different developers can work on different modules.

11. Layers in Software Architecture – Importance

- Layers in software architecture help organize the system into different levels, where each layer has a specific role.
- They improve structure, make the system easier to understand, and reduce complexity. Changes in one layer do not affect other layers much, which makes maintenance and updates easier.

12. Software Environments – Importance of Development Environment

- A development environment provides tools needed to write, test, and debug software.
- It includes code editors, compilers, debuggers, and libraries. A good development environment increases productivity, reduces errors, and helps developers build software efficiently and correctly.

13. Source Code – Difference between Source Code and Machine Code

- Source code is the human-readable code written by programmers using languages like C, Java, or Python.
- Machine code is the binary code (0s and 1s) that the computer understands and executes directly.
- In simple terms, source code is written by humans, while machine code is understood by computers.

14. GitHub and Introduction – Importance of Version Control

- Version control is important because it tracks changes in code over time.
- It allows developers to work together, manage different versions of a project, and easily revert to previous versions if something goes wrong. Tools like GitHub help in collaboration, backup, and maintaining code history.

15. Student Account in GitHub – Benefits for Students

Using GitHub provides many benefits for students in software development. It allows them to store their code online in repositories, making it accessible from anywhere. Students can track changes in their code, which helps in understanding progress and fixing mistakes easily. GitHub also supports collaboration, allowing multiple users to work on the same project. It helps students build a strong portfolio by showcasing their projects to recruiters. Additionally, students can contribute to open-source projects, which improves their coding skills and real-world experience.

16. Types of Software – Open-source vs Proprietary Software

Open-source software is software whose source code is freely available to the public. Users can modify, improve, and distribute it. This type of software encourages learning, transparency, and collaboration. Examples include

Linux and many free tools.

Proprietary software is owned by a company or individual, and its source code is not available to users. It requires a license to use, and users cannot modify it. Examples include many commercial applications. Open-source software is flexible and cost-effective, while proprietary software provides official support and better control over security.

17. GIT and GITHUB Training – Role of Git in Collaboration

Git is a powerful version control system that helps teams collaborate efficiently. It allows multiple developers to work on the same project without overwriting each other's work. Developers can create branches to work on different features independently and then merge them into the main project. Git tracks every change made in the code, which helps in identifying errors and reverting to previous versions if necessary. It improves teamwork, maintains project history, and ensures smooth collaboration in software development.

18. Application Software – Role in Businesses

Application software plays a vital role in business operations by helping perform specific tasks such as accounting, inventory management, communication, and customer relationship management. It increases efficiency by automating repetitive tasks, reduces human errors, and saves time. Businesses use application software to analyze data, generate reports, and make better decisions. It also improves communication within the organization and with customers, leading to overall productivity and growth.

19. Software Development Process – Main Stages

The software development process consists of several important stages that ensure the successful creation of software. It begins with planning, where goals and objectives are defined. Then comes requirement analysis, where user needs are identified. In the design phase, the structure and architecture of the system are planned. Development involves writing the actual code. Testing is done to identify and fix errors. Deployment is the process of releasing the software to users. Finally, maintenance ensures the software continues to work properly and is updated when needed.

20. Software Requirement – Importance of Requirement Analysis

Requirement analysis is one of the most critical phases in software development because it defines what the system should do. It involves gathering and analyzing user needs and expectations. A clear understanding of requirements helps developers build the correct system and avoid unnecessary changes later. Poor requirement analysis can lead to errors, increased costs, and project failure. Therefore, it ensures that the final product meets user expectations and works efficiently.

21. Software Analysis – Role in Development

Software analysis involves examining and understanding the requirements in detail before development begins. It helps in identifying system functionalities, constraints, and possible solutions. This phase ensures that developers clearly understand the problem and design an appropriate solution. It also helps in minimizing risks and improving the quality of the final software by planning everything carefully.

22. System Design – Key Elements

System design is the process of defining how a software system will be built. It includes designing the system architecture, data structures, user interfaces, and system components. It also defines how different parts of the system will interact with each other. A well-designed system improves performance, scalability, and maintainability. It acts as a blueprint for developers during the coding phase.

23. Software Testing – Importance

Software testing is essential to ensure that the software functions correctly and is free from bugs. It involves checking the system against requirements to verify its accuracy. Testing improves the quality, reliability, and security of the software. It helps identify errors before the software is released to users, reducing the risk of failure and improving user satisfaction.

24. Maintenance – Types of Software Maintenance

Software maintenance is performed after deployment to keep the system working effectively. There are four main types: corrective maintenance (fixing errors), adaptive maintenance (updating software for new environments or technologies), perfective maintenance (enhancing performance and features), and preventive maintenance (making improvements to avoid future problems). Maintenance ensures the software remains useful and up to date.

25. Development – Web vs Desktop Applications

Web applications run in a web browser and require an internet connection. They can be accessed from any device and are easy to update. Desktop applications are installed on a computer and can work offline. They are generally faster and provide better performance but are limited to a specific system. Web apps offer flexibility, while desktop apps offer power and speed.

26. Web Application – Advantages

Web applications provide several advantages such as accessibility from anywhere, no need for installation, and easy maintenance. Updates are done on the server side, so users always access the latest version. They are cost-effective and work across multiple platforms, making them highly flexible and user-friendly.

27. Designing – Role of UI/UX Design

UI (User Interface) and UX (User Experience) design play a crucial role in application development. UI focuses on the visual appearance, while UX focuses on usability and user satisfaction. Good UI/UX design makes applications easy to use, attractive, and efficient. It improves user engagement and ensures that users can interact with the system smoothly.

28. Mobile Application – Native vs Hybrid Apps

Native applications are developed for a specific platform such as Android or iOS and provide better performance, speed, and user experience. Hybrid applications are built using web technologies and can run on multiple platforms with a single codebase. While hybrid apps are cost-effective and easier to develop, they may not perform as efficiently as native apps.

29. DFD (Data Flow Diagram) – Significance

A Data Flow Diagram is used to represent how data flows through a system. It shows the input, processing, output, and storage of data. DFDs help in understanding system processes clearly and are widely used in system analysis and design. They simplify complex systems and improve communication between developers and users.

30. Desktop Application – Pros and Cons

Desktop applications offer high performance, reliability, and the ability to work offline. However, they require installation and are platform-dependent, meaning they may not run on all systems. They are powerful but less flexible compared to web applications, which are easier to access and update.

31. Flow Chart – Role in Programming and System Design

Flowcharts are graphical representations of a process or algorithm using symbols. They help in planning program logic, understanding complex processes, and identifying errors before coding. Flowcharts improve clarity and communication among developers and are especially useful for beginners.

